Université Lyon 1

Institut des Sciences Financières et d'Assurance

Développement d'une application en architecture 3-tiers: base de données de jeux Steam

Celia IMAKHLOUFEN

Master 2 *Data Sécurité Informatique*

November 29, 2025

Abstrait

Ce rapport présente la conception et le développement d'une application web suivant une architecture 3-tiers. L'objectif principal de ce projet est de mettre en pratique les concepts fondamentaux des architectures logicielles distribuées, en séparant clairement les couches de présentation, de logique métier et de données.

L'application est construite autour de technologies modernes et largement utilisées dans l'industrie. La couche de présentation utilise JavaScript pour offrir une interface utilisateur interactive et réactive. La couche métier est implémentée en Python, assurant le traitement des requêtes et l'application des règles fonctionnelles. Enfin, MongoDB, une base de données NoSQL orientée documents, est employée pour la persistance des données, offrant flexibilité et scalabilité.

Ce projet met en évidence les avantages de l'architecture 3-tiers, notamment la modularité, la maintenabilité et la possibilité de faire évoluer chaque couche indépendamment. Nous détaillons les choix techniques effectués, les défis rencontrés lors de l'implémentation, ainsi que les solutions adoptées pour garantir la cohérence et la performance du système.

Les résultats obtenus démontrent l'efficacité de cette approche architecturale pour le développement d'applications web robustes et évolutives, tout en respectant les bonnes pratiques de génie logiciel.

Mots-clés: architecture 3-tiers, base de données, JavaScript, Python, MongoDB

Contents

List of Figures iii

List of Tables iv

1 Introduction 1
- 1.1 Contexte . . . 1
- 1.2 Problématique . . . 1
- 1.3 Buts et objectifs . . . 2
- 1.4 Solution . . . 3
  - 1.4.1 Architecture 3-tiers . . . 3
  - 1.4.2 Technologies et outils . . . 3
  - 1.4.3 Méthodologie de développement . . . 3
- 1.5 Résultats . . . 4
- 1.6 Organisation du rapport . . . 5

References 6

Appendices 7

A An Appendix Chapter (Optional) 7

B An Appendix Chapter (Optional) 8

List of Figures

List of Tables

List of Abbreviations

SMPCS	School of Mathematical, Physical and Computational Sciences

Chapter 1

Introduction

1.1 Contexte

La gestion et l'exploration de grandes bases de données constituent un défi majeur dans le domaine des systèmes d'information modernes. Avec la croissance exponentielle des données numériques, notamment dans l'industrie du jeu vidéo, il devient essentiel de développer des architectures logicielles robustes et scalables permettant de traiter efficacement des volumes de données importants tout en offrant une expérience utilisateur fluide et performante.

Steam, la plateforme de distribution numérique de jeux vidéo développée par Valve Corporation, compte aujourd'hui plus de 100 000 jeux dans son catalogue. Cette volumétrie importante pose des problématiques techniques intéressantes en termes de stockage, d'indexation, de recherche et de filtrage des données. L'exploitation de ces métadonnées nécessite une architecture logicielle appropriée capable de gérer efficacement les opérations de lecture, d'écriture et de recherche sur un volume conséquent de documents.

L'architecture 3-tiers représente un modèle de conception logicielle largement adopté dans le développement d'applications web modernes. Ce pattern architectural divise une application en trois couches distinctes et indépendantes : la couche de présentation (interface utilisateur), la couche métier (logique applicative) et la couche de données (persistance). Cette séparation des préoccupations offre plusieurs avantages significatifs, notamment une meilleure maintenabilité du code, une réutilisabilité accrue des composants et une scalabilité facilitée.

Dans le contexte actuel où les bases de données NoSQL, et notamment MongoDB, gagnent en popularité pour le stockage de données semi-structurées, il devient pertinent d'explorer comment intégrer ces technologies dans une architecture 3-tiers classique. MongoDB offre des avantages notables pour le stockage de métadonnées de jeux vidéo grâce à sa flexibilité schématique, ses capacités d'indexation avancées et ses performances en lecture sur de gros volumes de données.

1.2 Problématique

La manipulation et l'exploration d'une base de données volumineuse de jeux vidéo (plus de 100 000 entrées) soulèvent plusieurs défis techniques et architecturaux qu'il convient d'adresser de manière rigoureuse.

Premièrement, le stockage et la récupération efficace de données semi-structurées nécessitent une base de données adaptée. Les métadonnées des jeux Steam présentent une structure hétérogène avec des champs optionnels, des tableaux de tailles variables (genres, tags) et des données textuelles de longueurs diverses. Une base de données relationnelle traditionnelle

imposerait un schéma rigide peu adapté à cette variabilité.

Deuxièmement, les performances de recherche et de filtrage doivent rester acceptables malgré le volume important de données. Les utilisateurs s'attendent à pouvoir effectuer des recherches textuelles, filtrer par genre, trier selon différents critères (popularité, prix, date de sortie) et paginer les résultats de manière fluide. Sans optimisation appropriée (indexation, requêtes efficaces), ces opérations pourraient devenir prohibitives en termes de temps de réponse.

Troisièmement, l'architecture logicielle doit permettre une séparation claire des responsabilités. Le couplage fort entre l'interface utilisateur, la logique métier et l'accès aux données rend le code difficile à maintenir, à tester et à faire évoluer. Une architecture bien structurée est nécessaire pour garantir la pérennité et l'évolutivité de l'application.

Enfin, l'application doit offrir des opérations CRUD (Create, Read, Update, Delete) complètes avec une persistance fiable des données. Les modifications apportées par les utilisateurs administrateurs doivent être immédiatement reflétées dans la base de données et visibles pour tous les clients connectés.

1.3 Buts et objectifs

Aim principal : Concevoir et implémenter une application web de gestion et d'exploration de jeux vidéo Steam en utilisant une architecture 3-tiers avec MongoDB comme système de gestion de base de données.

Objectifs spécifiques :

1. **Conception architecturale :** Implémenter une architecture 3-tiers stricte avec séparation claire entre la couche de présentation (frontend HTML/CSS/JavaScript), la couche métier (GameService) et la couche de données (GameDatabase avec MongoDB).

2. **Intégration MongoDB :** Mettre en place une base de données MongoDB optimisée pour stocker et interroger efficacement plus de 100 000 documents de jeux, incluant la création d'index appropriés pour les recherches textuelles et les tris.

3. **Fonctionnalités de recherche avancées :** Développer un système de recherche complet permettant le filtrage par genre, la recherche textuelle sur titres et tags, le tri multi-critères (titre, score, prix, date, nombre d'avis, popularité) et la pagination des résultats.

4. **Opérations CRUD complètes :** Implémenter l'ensemble des opérations de création, lecture, mise à jour et suppression de jeux avec persistance dans MongoDB et validation des données.

5. **Import/Export de données :** Fournir des fonctionnalités d'import et d'export au format CSV pour faciliter la migration et la sauvegarde des données.

6. **Interface utilisateur ergonomique :** Créer une interface web moderne et responsive offrant une expérience utilisateur fluide avec affichage des statistiques en temps réel, prévisualisation des images et panel d'administration.

7. **API REST documentée :** Exposer une API REST complète et cohérente respectant les bonnes pratiques HTTP pour permettre l'intégration avec d'autres systèmes.

1.4 Solution

Pour répondre aux objectifs fixés, nous avons adopté une approche méthodologique structurée autour de l'architecture 3-tiers et des technologies web modernes.

1.4.1 Architecture 3-tiers

L'application a été conçue selon le modèle architectural 3-tiers comme demandé, garantissant une séparation stricte des préoccupations :

- **Couche de présentation** : Interface utilisateur développée en HTML5, CSS3 et JavaScript vanilla. Cette couche gère l'affichage des jeux, les interactions utilisateur, la pagination et communique avec la couche métier via une API REST.
- **Couche métier** : Implémentée par la classe `GameService`, cette couche contient toute la logique applicative incluant le filtrage des jeux, les calculs de statistiques, la validation des données et les transformations nécessaires. Elle fait l'interface entre la couche de présentation et la couche de données.
- **Couche de données** : Réalisée par la classe `GameDatabase`, cette couche encapsule tous les accès à MongoDB. Elle expose des méthodes pour les opérations CRUD, la gestion de la connexion et la création d'index, isolant ainsi complètement la logique de persistance.

1.4.2 Technologies et outils

Le choix technologique s'est porté sur un stack JavaScript full-stack :

- **Backend** : Node.js (v20.12.2) avec Express.js pour le serveur HTTP et le routage des requêtes. Le driver natif MongoDB (v7.0.0) assure la communication avec la base de données.
- **Base de données** : MongoDB (v7.0+) a été sélectionné pour sa flexibilité schématique adaptée aux données semi-structurées, ses capacités d'indexation avancées (index textuels, index composites) et ses performances en lecture sur de gros volumes.
- **Frontend** : Technologies web standards (HTML5, CSS3, JavaScript ES6+) avec l'API Fetch pour les communications asynchrones avec le backend. Aucun framework lourd n'a été utilisé pour maintenir la légèreté et la compréhension du code.
- **Import de données** : JSONStream pour le parsing efficace du fichier de métadonnées de 92 Mo contenant 109 632 jeux, permettant un traitement en streaming pour éviter la saturation mémoire.

1.4.3 Méthodologie de développement

Le développement a suivi une approche itérative en plusieurs phases :

1. **Phase 1** : Conception de l'architecture et mise en place de la structure du projet avec séparation claire des couches.
2. **Phase 2** : Implémentation de la couche de données avec MongoDB, incluant l'import des métadonnées Steam et la création d'index optimisés.

3. **Phase 3** : Développement de la couche métier avec implémentation des algorithmes de recherche, filtrage et tri.
4. **Phase 4** : Création de l'API REST et de la couche de présentation avec interface utilisateur responsive.
5. **Phase 5** : Ajout des fonctionnalités avancées (CRUD complet, import/export CSV, statistiques en temps réel).
6. **Phase 6** : Tests, optimisations et documentation du projet.

1.5 Résultats

Ce projet a permis de réaliser avec succès une application web complète de gestion de jeux vidéo Steam, démontrant l'efficacité de l'architecture 3-tiers combinée à MongoDB pour la manipulation de grandes quantités de données.

Contributions principales :

- **Architecture robuste** : Implémentation d'une architecture 3-tiers stricte et bien documentée, servant de référence pour le développement d'applications web scalables. La séparation des couches permet une maintenabilité accrue et facilite les évolutions futures.
- **Base de données optimisée** : Mise en place d'une base MongoDB contenant 109 632 jeux avec indexation stratégique (index textuels, index sur genres, index sur métadonnées numériques) garantissant des temps de réponse inférieurs à 100ms pour la majorité des requêtes.
- **Système de recherche performant** : Développement d'un moteur de recherche multi-critères supportant la recherche textuelle full-text, le filtrage par genre, le tri selon six critères différents et la pagination, le tout fonctionnant efficacement sur l'ensemble du catalogue.
- **API REST complète** : Exposition de 8 endpoints REST couvrant toutes les opérations CRUD, les statistiques, l'export/import CSV et la recherche avancée, respectant les conventions HTTP et retournant des réponses JSON structurées.
- **Interface utilisateur moderne** : Création d'une interface web responsive avec affichage de cartes de jeux enrichies (images, scores, métadonnées), panel d'administration pour la gestion des données et statistiques dynamiques.
- **Fonctionnalités d'import/export** : Implémentation de fonctionnalités CSV permettant l'export de données filtrées et l'import en masse de jeux avec gestion des erreurs et rapport détaillé.

Résultats mesurables :

- Application capable de gérer et d'afficher plus de 100 000 jeux avec des performances constantes
- Temps de chargement initial inférieur à 500ms
- Pagination fluide avec 20 jeux par page

- Recherche textuelle avec résultats instantanés
- Support de l'import CSV avec traitement de plusieurs milliers de lignes
- Architecture facilement extensible pour l'ajout de nouvelles fonctionnalités

Les résultats obtenus démontrent que l'architecture 3-tiers associée à MongoDB constitue une solution pertinente pour le développement d'applications web manipulant de grands volumes de données semi-structurées, tout en maintenant une séparation claire des responsabilités et des performances satisfaisantes.

1.6 Organisation du rapport

Ce rapport est structuré en plusieurs chapitres présentant de manière détaillée la conception, l'implémentation et l'évaluation du projet.

Le **Chapitre ??** présente une revue de littérature couvrant les concepts fondamentaux de l'architecture 3-tiers, les bases de données NoSQL et particulièrement MongoDB, ainsi qu'un état de l'art des technologies web modernes utilisées dans le projet. Cette revue permet de situer le projet dans son contexte technique et théorique.

Le **Chapitre ??** détaille la méthodologie adoptée pour la conception et le développement de l'application. Il décrit l'architecture globale du système, les choix technologiques effectués, la structure des données et la modélisation de la base MongoDB.

Le **Chapitre ??** expose l'implémentation concrète de chacune des trois couches de l'architecture. Il présente le code des composants principaux (`GameDatabase`, `GameService`, `GameController`), les algorithmes de recherche et de filtrage, ainsi que l'interface utilisateur.

Le **Chapitre ??** analyse les résultats obtenus, incluant des mesures de performance, des tests de scalabilité et une évaluation de l'efficacité des index MongoDB. Ce chapitre présente également des captures d'écran de l'application en fonctionnement.

Le **Chapitre ??** discute les forces et limitations du projet, compare l'approche retenue avec des alternatives possibles, et propose des pistes d'amélioration et d'évolution de l'application.

Enfin, le **Chapitre ??** conclut le rapport en synthétisant les contributions du projet, en rappelant dans quelle mesure les objectifs ont été atteints, et en ouvrant sur des perspectives de recherche et de développement futur.

Les **Annexes** fournissent des informations complémentaires telles que le guide d'installation complet, la documentation de l'API REST, des extraits de code détaillés et le manuel utilisateur de l'application.

References

Appendix A

An Appendix Chapter (Optional)

Some lengthy tables, codes, raw data, length proofs, etc. which are **very important but not essential part** of the project report goes into an Appendix. An appendix is something a reader would consult if he/she needs extra information and a more comprehensive understating of the report. Also, note that you should use one appendix for one idea.

An appendix is optional. If you feel you do not need to include an appendix in your report, avoid including it. Sometime including irrelevant and unnecessary materials in the Appendices may unreasonably increase the total number of pages in your report and distract the reader.

Appendix B

An Appendix Chapter (Optional)

...